MODULE-1

Introduction to Databases: Introduction, Characteristics of the database approach, Actors on the scene, Workers behind the scene, Advantages of using DBMS Approach.

Database System Concepts and Architecture: Data models, schemas and instances, Three-schema architecture and Data independence.

Data Modelling using the Entity-Relationship (ER) Model: Entity types, Entity sets, Attributes and keys, Relationship types, Relationship sets, Roles and Structural Constraints, Weak Entity Types, Refining the ER diagram of the COMPANY Database, ER diagrams, Naming Conventions and Design issues, Relational database design using ER-to-Relational Mapping.

Introduction to Databases

- Introduction
- Characteristics of the database approach,
- Actors on the scene,
- Workers behind the scene,
- Advantages of using DBMS Approach.

INTRODUCTION

Data : Facts those are recorded for an objects.

- For example, data can be of text, numbers, video etc..

A **Database** is a collection of **related data** and used to organize data in form of table, views, Reports

- For example, consider the names, telephone numbers, and addresses of the people you know.
- Considering a university need a database to store details about **Teachers, Student, Courses, Exams.**



INTRODUCTION

- A **database management system (DBMS)** that enables users to create and maintain a database.
- The DBMS is a collection of programs or *software system* that facilitates the processes of ***defining, constructing,*** **and** ***manipulating*** databases for various applications.

Basic Functions of DBMS:

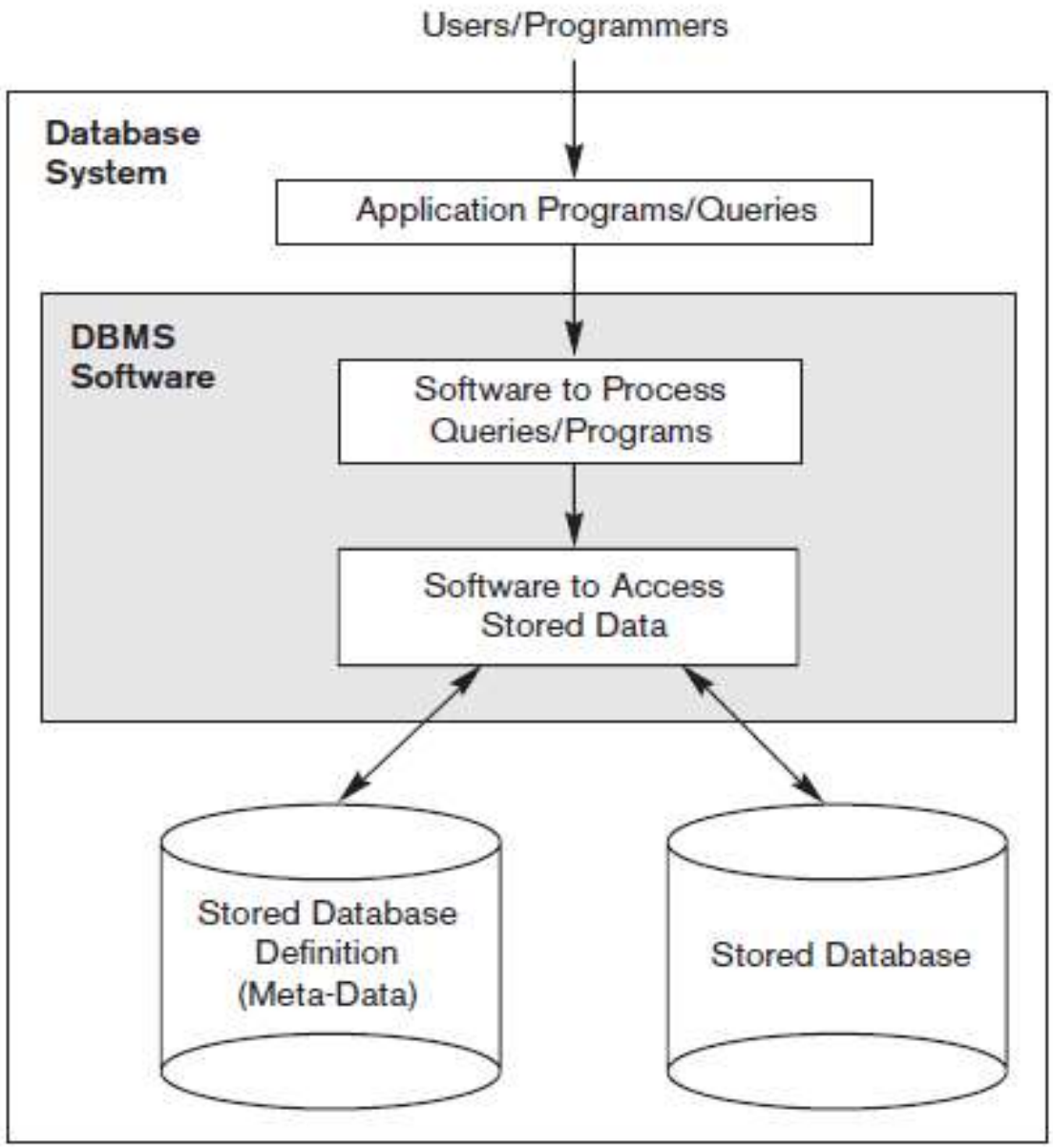
- To store data in database
- To update the data
- To delete the data
- To access the data



INTRODUCTION

- **Defining** a database involves specifying the data types, structures, and constraints for the data to be stored in the database.
- **Constructing** the database is the process of storing the data itself on some storage medium that is controlled by the DBMS.
- **Manipulating** a database includes such functions as querying the database to retrieve specific data, updating the database to reflect changes in the Mini world, and generating reports from the data.
- **Sharing** a database allows multiple users and programs to access the database simultaneously.
- **Application programs** access the database by sending queries or request for data to the DBMS.A query causes some data to be retrieved

A SIMPLIFIED DATABASE SYSTEM ENVIRONMENT.



Characteristics of database approach

1. Self-Describing Nature of a Database System.
2. Insulation between Programs and Data, and Data Abstraction.
3. Support of Multiple Views of the Data.
4. Sharing of Data and Multiuser Transaction Processing.



CHARACTERISTICS OF DATABASE APPROACH

1. Self-Describing Nature of a Database System:

- A **Database Management System (DBMS)** is considered *self-describing* because it stores both the actual data and its structural definition, known as **metadata**.
- Metadata includes details such as table structures, data types, constraints, relationships, and indexing information, all of which help the DBMS understand how to organize and manage the data independently of any specific application.
- This allows the DBMS to work independently of specific applications, enhancing flexibility and scalability.



CHARACTERISTICS OF DATABASE APPROACH

Key Components of Self-Describing Nature

1. **Metadata (Data About Data)**
    1. The DBMS maintains a catalog (also called a data dictionary or system catalog) that contains metadata.
    2. This metadata describes:
        1. **Tables and their schemas** (column names, data types, constraints)
        2. **Relationships** between tables (foreign keys, primary keys)
        3. **Indexes** for optimizing search queries
        4. **Access controls** (user permissions, roles)
2. **Independence from Applications**
    1. Unlike traditional file-based systems, where structure is tightly coupled with the application code, a DBMS allows applications to interact with the data without needing to know its underlying structure explicitly.
    2. Any changes in the schema (such as adding a new column) can be reflected dynamically through the catalog, without requiring modifications in all dependent applications.
3. **Enhanced Flexibility and Scalability**
    1. Since the database structure is stored and managed separately, different applications can access and manipulate the same data without redundancy.
    2. The DBMS can optimize query processing based on the stored metadata.

CHARACTERISTICS OF DATABASE APPROACH

1. Actual Data (Stored in Tables)
Students Table

Student_ID	Name	Age	Course
101	Alice	21	Computer Science
102	Bob	22	Mathematics

2. Metadata (Stored in System Catalog)
The DBMS stores the following metadata in a system catalog:
Table Name: Students
Columns: -

Student_ID (Integer, Primary Key)
Name (Varchar, Max 50 Characters)
Age (Integer, Constraint: Age > 17)
Major (Varchar, Max 50 Characters)



CHARACTERISTICS OF DATABASE APPROACH

2. Insulation between Programs and Data, and Data Abstraction:

- In a DBMS, the structure of data files is separated from application programs through the database catalog.
- This separation, known as program-data independence, allows changes in database structure without altering the programs that access the data.
- Data abstraction provides a simplified, conceptual representation of the data, hiding details about data storage and operations from the user.



Characteristics of database approach

3. Support of Multiple Views of the Data:

- A DBMS can provide different views of the data for different users, allowing customized views that meet specific user needs without altering the underlying database structure.
- These views can include subsets of the database or virtual data derived from the database.



CHARACTERISTICS OF DATABASE APPROACH

4. Sharing of Data and Multiuser Transaction Processing:

- A DBMS enables multiple users to access and modify the database concurrently without conflict, through concurrency control mechanisms.
- It supports complex transaction processing, ensuring that transactions are processed reliably and ensuring properties like atomicity (all-or-nothing execution) and isolation (independence of transactions).



Actors on the Scene

- The people whose jobs involve the day-to-day use of a large database;

1. Database Administrators
2. Database Designers
3. End Users
   - Casual end users.
   - parametric end users
   - Sophisticated end users
   - Stand-alone users
4. System Analysts and Application Programmers (Software Engineers)

Workers behind the Scene

Workers behind the scene are those who are associated with the design, development, and operation of the DBMS *software and system environment*. These persons are typically not interested in the database content.

1. DBMS system designers and implementers
2. Tool developers
3. Operators and maintenance personnel



Advantages of using the DBMS approach

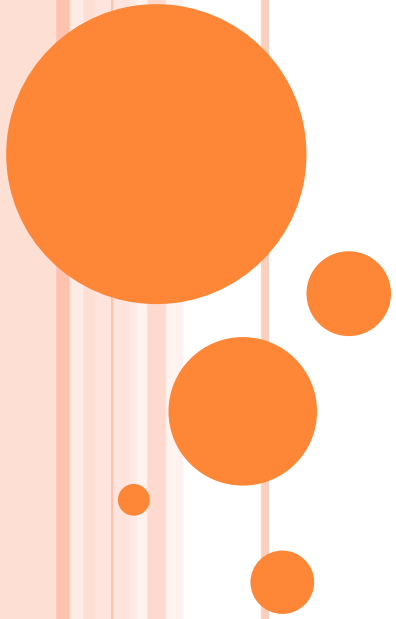
- Controlling Redundancy
- Restricting Unauthorized Access
- Providing Persistent Storage for Program Objects
- Providing Storage Structures and Search Techniques for Efficient Query Processing
- Providing Backup and Recovery
- Providing Multiple User Interfaces
- Representing Complex Relationships among Data
- Enforcing Integrity Constraints



Advantages of using the DBMS approach

Aspect	File System Approach	DBMS Approach
Data Redundancy	High data redundancy because the same data is stored in multiple files.	Redundancy is minimized since data is stored in a centralized database.
Data Consistency	Difficult to maintain consistency due to duplication of data.	Consistency is maintained automatically when data is updated.
Data Sharing	Limited sharing of data among different users and applications.	Multiple users can access and share data simultaneously.
Security	Provides limited security and access control.	Provides strong security with authentication and authorization mechanisms.
Data Integrity	Integrity rules cannot be enforced effectively.	Integrity constraints (primary key, foreign key, etc.) ensure accurate data.
Backup and Recovery	Backup and recovery processes are manual and complex.	Automatic backup and recovery mechanisms are available.
Data Independence	No data independence; programs depend on file structure.	Provides logical and physical data independence.
Query Processing	Requires custom programs to retrieve data.	Uses query languages like SQL for efficient data retrieval.
Concurrent Access	Difficult to manage multiple users accessing data at the same time.	Concurrency control allows safe multi-user access.
Data Management	Data is scattered across multiple files and systems.	Data is centrally managed by a Database Management System.

Database System Concepts and Architecture



Data Model

- A **data model :** a collection of concepts that can be used to describe the structure of a database.
- By *structure of a database,* we mean the **data types, relationships**, and **constraints** that should hold on the data.
- Most data models also include a set of **basic operations** for specifying retrievals and updates on the database.
- Also includes concepts to specify the behaviour of a database application.



Categories of Data Models

A) High-level or **conceptual data models** provide concepts that are close to the way many users perceive data.

- Conceptual data model use concepts such as entities, attributes and relationship
- An **entity** represents a real-world object or concept, such as an employee or a project, that is described in the database.
- An **attribute** represents some property of interest that further describes an entity, such as the employee's name or salary.
- A **relationship** among two or more entities represents an interaction among the entities; for example, a works-on relationship between an employee and a project. Representational or implementation data models are the models used most frequently in traditional commercial DBMSs, and they include the widely-used **relational data model.**

Categories of Data Models

B) Low-level or **physical data models** provide concepts that describe the details of how data is stored in the computer.

C) Representational (or **implementation**) **data models,** which provide concepts that may be understood by end users but that are not too far removed from the way data is organized within the computer.

- Representational data models represent data by using record structures and hence are sometimes called **record-based data models.**
- Representational or implementation data models are the models used most frequently in traditional commercial DBMSs, and they include the widely-used relational data model. As well as in Legacy data models the **network and hierarchica**l models.



Schemas, Instances, and Database State.

STUDENT

Name	Student_number	Class	Major

COURSE

Course_name	Course_number	Credit_hours	Department

PREREQUISITE

Course_number	Prerequisite_number

SECTION

Section_identifier	Course_number	Semester	Year	Instructor

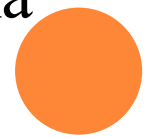
GRADE_REPORT

Student_number	Section_identifier	Grade

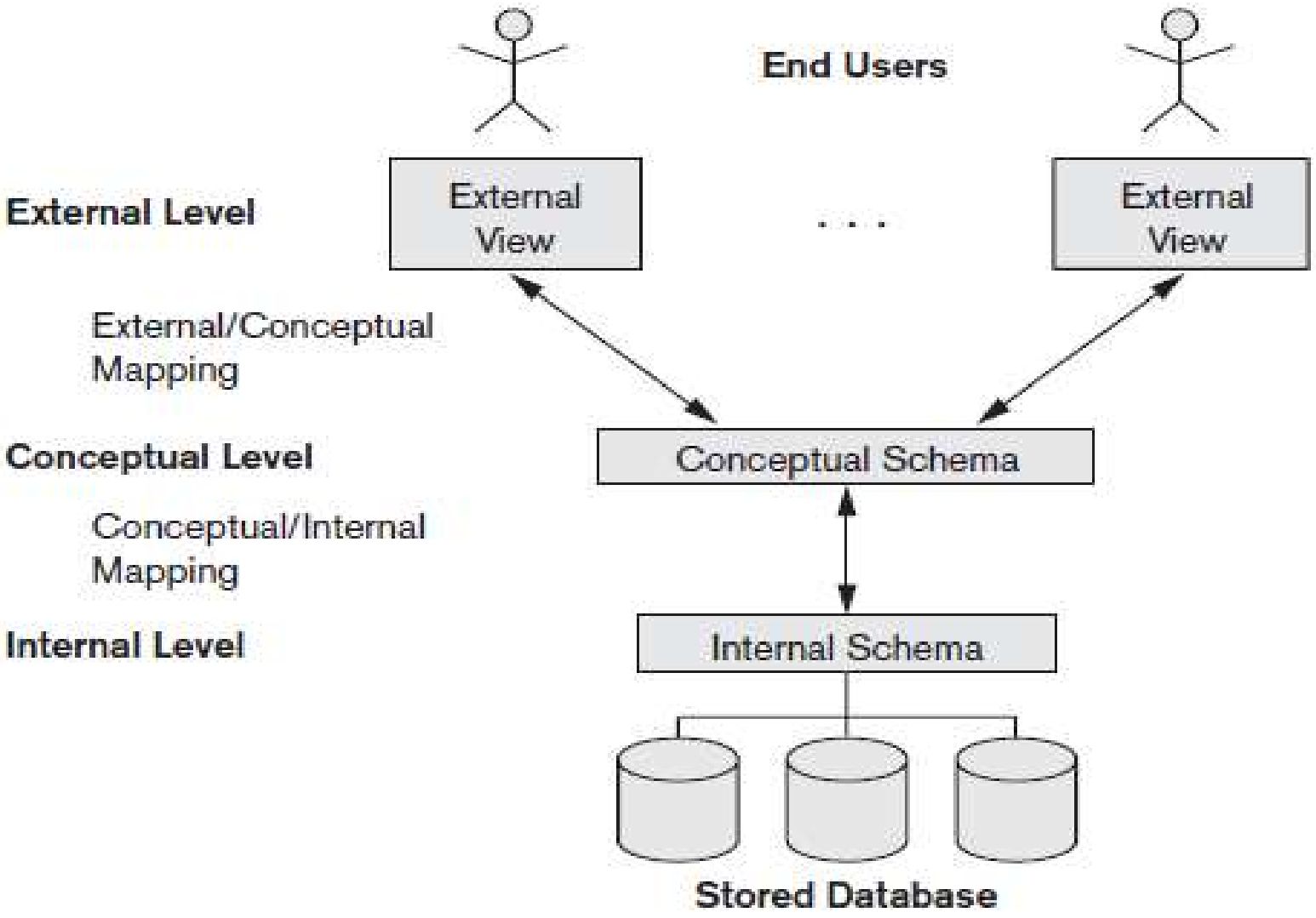
Schema diagram for a database

Schemas, Instances, and Database State.

- The data in the database at a particular moment in time is called a **database state** or **snapshot**. It is also called the current set of **occurrences** or **instances** in the database.
- In a given database state, each schema construct has its own current set of instances;
- The DBMS is partly responsible for ensuring that every state of the database is a **valid state** that is, a state that satisfies the structure and constraints specified in the schema.
- The DBMS stores the descriptions of the schema constructs and constraints also called the **meta-data** in the DBMS catalog so that DBMS software can refer to the schema whenever it needs to.



THREE-SCHEMA ARCHITECTURE



The three-schema architecture.

Three-Schema Architecture and Data Independence

The goal of the three-schema architecture is to separate the user applications from the physical database. In this architecture, schemas can be defined at the following three levels:

1. The internal level has an internal schema, which describes the physical storage structure of the database.

- The internal schema uses a physical data model and describes the complete details of **data storage** and **access paths** for the database.



Three-Schema Architecture and Data Independence

2. The conceptual level has a conceptual schema, which describes the structure of the whole database for a community of users.

- The conceptual schema hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints.
- A representational data model is used to describe the conceptual schema when a database system is implemented.

3. The external or view level includes a number of **external schemas** or **user views**. Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group.



Data Independence

The ability to modify a scheme definition in one level without affecting a scheme definition in a higher level is called **data independence**

The two types of data independence are:

1. Logical data independence is the capacity to change the conceptual schema without having to change external schemas or application programs.

- The conceptual schema could be changed to expand the database (by adding a record type or data item), to change constraints, or to reduce the database (by removing a record type or data item).
- The view definition and the mappings need to be changed in a DBMS that supports logical data independence.
- After the conceptual schema undergoes a logical reorganization, application programs that reference the external schema constructs must work as before. Changes to constraints can be applied to the conceptual schema without affecting the external schemas or application programs.



Data Independence

Scenario: Online Retail Store Database

Imagine an online store like Amazon that keeps track of customers, orders, and products in a database. The store's website and mobile app use this database to show customers their orders.

Change in Logical Structure Without Affecting Applications

At first, the Orders table in the database looks like this:

Orders table

Order_ID	Customer_ID	Product_ID	Order_Date

Later, to improve efficiency, the company splits the table into two:

Orders Table → (Order_ID, Customer_ID, Order_Date)

Order_Details Table → (Order_ID, Product_ID, Quantity, Price)



Data Independence

2. Physical data independence is the capacity to change the internal schema without having to change the conceptual schema. Hence, the external schemas need not be changed as well.

- Changes to the internal schema may be needed because some physical files were reorganized for example, by creating additional access structures to improve the performance of retrieval or update.
- If the same data as before remains in the database, we should not have to change the conceptual schema.



Data Independence

Scenario: Online Retail Store Database

- Imagine an **e-commerce website** like **Amazon**, which has a database storing **Customers, Orders, and Products**. The mobile app and website fetch this data to show users their order history, product details, etc.
- **Change in Physical Storage Without Affecting Applications**
- At first, the company stores all its data on **a single server** with a hard drive. Later, to improve performance, the database administrator makes **physical storage changes**:

1. **Moves the database to a cloud server (AWS, Google Cloud, etc.).**
2. **Changes storage from HDD to SSD for faster performance.**
3. **Reorganizes the way data is indexed for quick retrieval.**

Data Modelling using the Entity-Relationship (ER) Model:

Entity types, entity sets, attributes and keys, relationship types, relationship sets, roles and structural constraints, weak entity types, refining the ER diagram of the COMPANY database, ER diagrams, naming conventions and design issues, relational database design using ER-to-relational mapping.



Entity Types, Entity Sets, Attributes and Keys

Entities and Attributes.

- The basic concept of an ER diagram is the **ENTITY**.
- An **Entity** represents a **'thing'** or **'object'** in the real world.
- Examples of entities might be an object with physical existence, such as a student, a house, a product etc, or conceptual entities such as a company, a job position, a course, etc.
- **Entities** have **attributes**, which basically are the properties/ characteristics of a particular entity.



Entity Types, Entity Sets, Attributes and Keys

Entity Types:

- An Entity Type defines a collection or set of entities that have the same attributes. **Entities** are the instances of people, places, things, or events that are of interest.
- Each entity type in the database is described by its name and attributes.
- The entity shares the same attributes, but each entity has its own value for each attribute.

Entity Set:

- The collection of all entities of a particular entity type in the database at any point in time is called an **entity set.**



Entity Types, Entity Sets, Attributes and Keys

- **Entity Type**: Student
- **Entity Set:**

Entity Set Example:

[123, John, Smith, 12/01/1981, Computer Technology]

[456, Jane, Doe, 05/02/1979, Mathematics]

[789, Semra, Aykan, 02/08/1980, Linguistics]



Entity Types, Entity Sets, Attributes and Keys

Types of attributes occur in the ER model:

1. Simple versus composite
2. Single valued versus multi valued
3. Stored versus derived.
4. Null Valued Attributes
5. Complex Attributes



Entity Types, Entity Sets, Attributes and Keys

1. Simple vs. Composite Attributes:

- Attributes that are not divisible into subparts are called **simple or atomic attributes.**
  - Eg: Age
- **Composite attributes** can be divided into smaller subparts, which represent more basic attributes, independent meanings.
  - Example of a composite attribute is Address. Address can be broken down into a number of subparts, such as Street Address, City, Postal Code. Street Address may be further broken down by Number, Street Name and Apartment/Unit number.



Entity Types, Entity Sets, Attributes and Keys

2. Single-Valued vs. Multi-valued Attributes

- The attributes that have a single value for a particular entity, such a attribute are called Single valued.
  - For example Age is a single valued attribute of a person.
- An attribute can have multiple values for a single entity; these attributes are called multi-valued attributes.
  - For example, a doctor may have more than one specialty (or may have only one specialty), a customer may have more than one mobile phone number, or they may not have one at all.



Entity Types, Entity Sets, Attributes and Keys

3. Stored vs. Derived Attributes

- The attribute that is used to derive the attribute is called a stored attribute
- If an attribute can be determined using the value of another attribute, they are called derived attributes.
- For a particular person entity, the value of Age can be determined from the current (today's) date and the value of that person's Birth_date.
- The Age attribute is hence called a derived attribute and is said to be derivable from the Birth_date attribute, which is called a stored attribute.
- Derived attributes are not stored in the file, but can be derived when needed from the stored attributes.



Entity Types, Entity Sets, Attributes and Keys

4. **Null Valued Attributes**

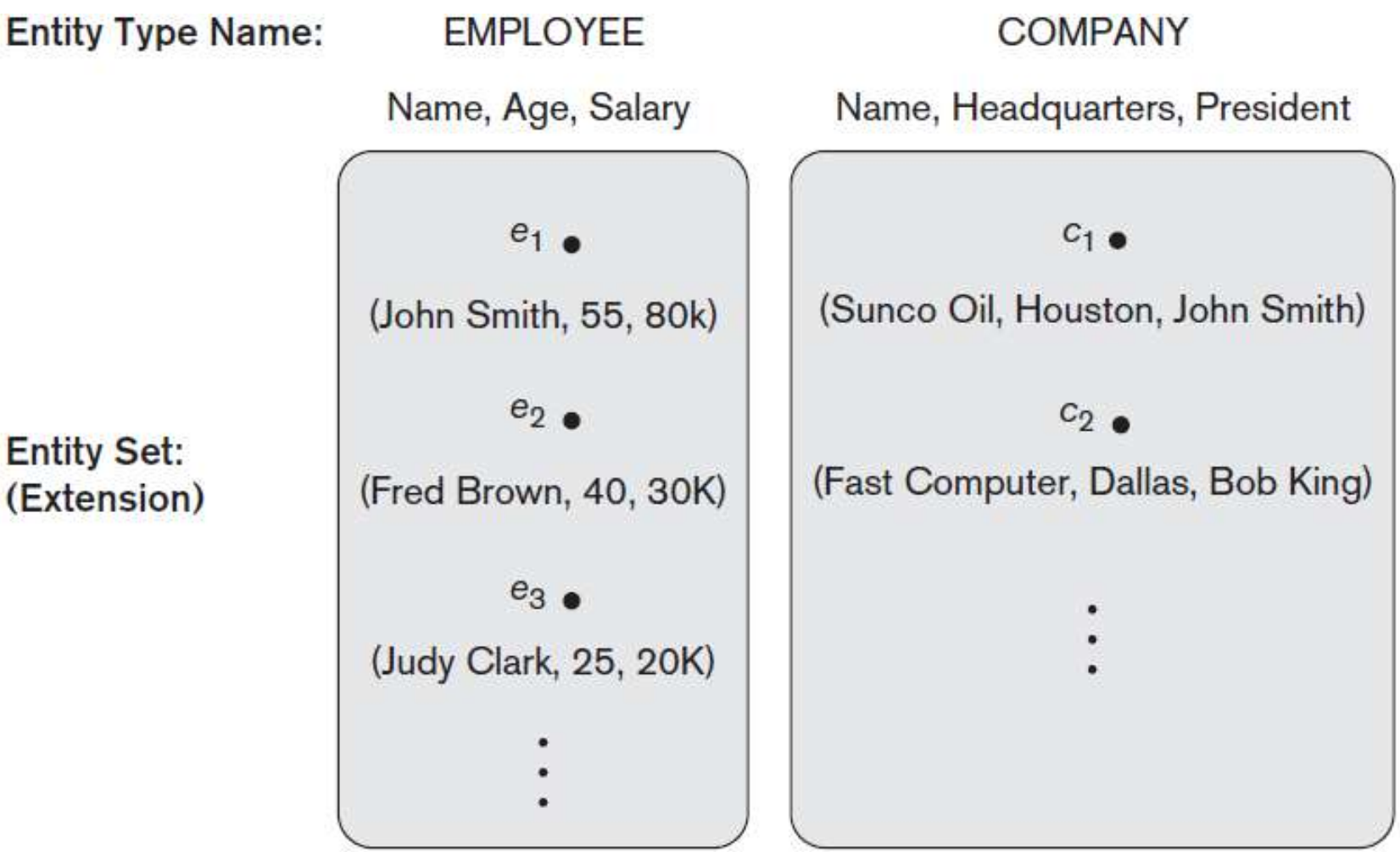
- There are cases where an attribute does not have an applicable value for an attribute. For these situations, the value null is created.
- A person who does not have a mobile phone would have null stored at the value for the Mobile Phone Number attribute.

5. **Complex Attributes**

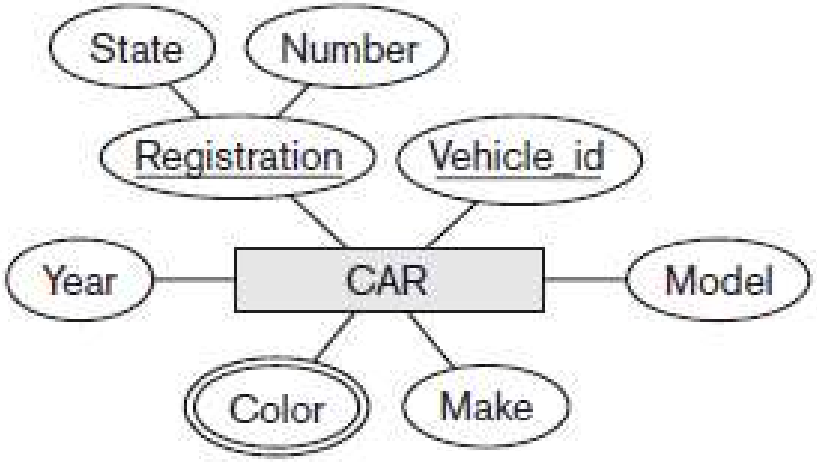
- Complex attributes are attributes that are nested in an arbitrary way.
- For example, a person can have more than one residence, and each residence can have more than one phone, therefore it is a complex attribute that can be represented as:
- The composite and multivalued attributes can be nested arbitrarily by grouping components of a composite attribute between parentheses () and separating the components with commas, and by displaying multivalued attributes between braces { }.Such attributes are called complex attributes.



Entity Types, Entity Sets, Attributes and Keys



Entity Types, Entity Sets, Attributes and Keys



CAR

Registration (Number, State), Vehicle_id, Make, Model, Year, {Color}

CAR_1

((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 2004 {red, black})

CAR_2

((ABC 123, NEW YORK), WP9872, Nissan Maxima, 4-door, 2005, {blue})

CAR_3

((VSY 720, TEXAS), TD729, Chrysler LeBaron, 4-door, 2002, {white, blue})

.
.
.

Entity Types, Entity Sets, Attributes and Keys

Key Attributes of an Entity Type.

- An important constraint on entities of an entity type is the uniqueness constraint.
- A **key attribute is an attribute whose values are distinct for each individual entity in the entity set.**
- The values of the key attribute can be used to identify each entity uniquely.
- ***Example: The Name attribute is a key of the COMPANY entity type because no two companies are allowed to have the same name. For the PERSON entity type, a typical key attribute is Usn.***
- In ER diagrammatic notation, each key attribute has its name **underlined inside the oval.**
- Those entities with no key attribute are **called weak entity types.**

Entity Types, Entity Sets, Attributes and Keys

Value Sets (Domains) of Attributes.

- Each simple attribute of an entity is associates with a domain of values, or value set, which specifies the set of values that may be assigned to that attribute for each entity.
- ***For example, date of birth must be before today's date, and after 01/01/1900***



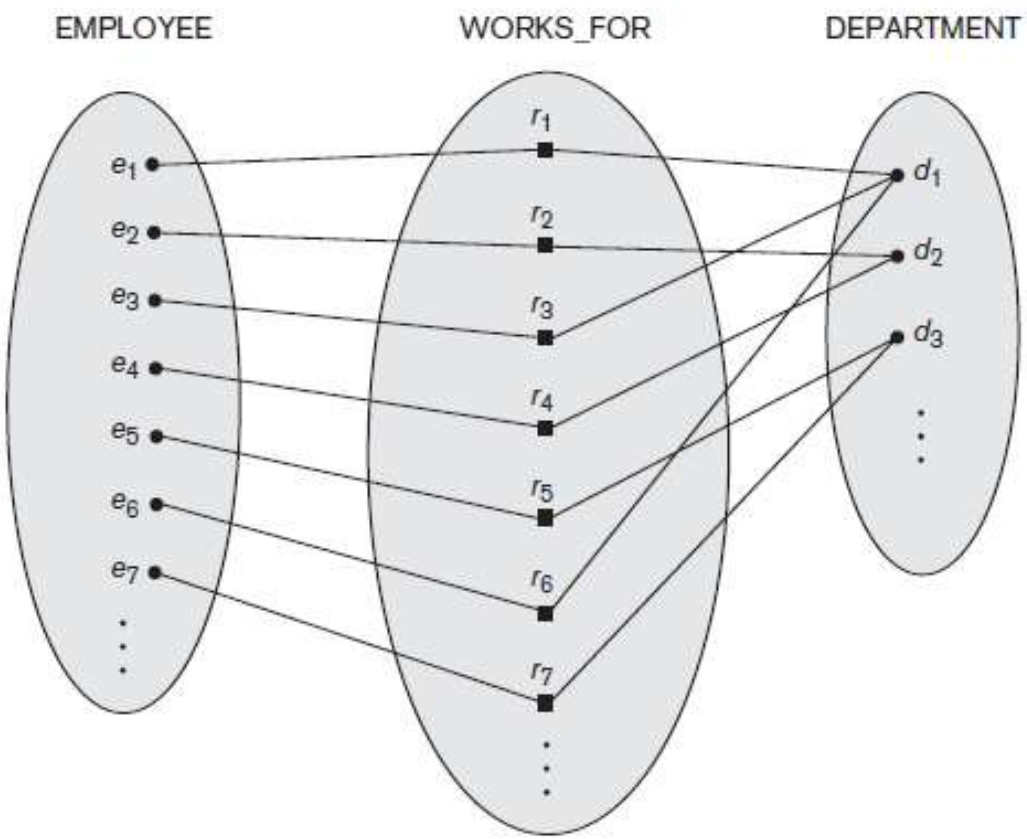
Relationship Types, Relationship Sets, Roles, & Structural Constraints

Relationship Types, Sets, and Instances

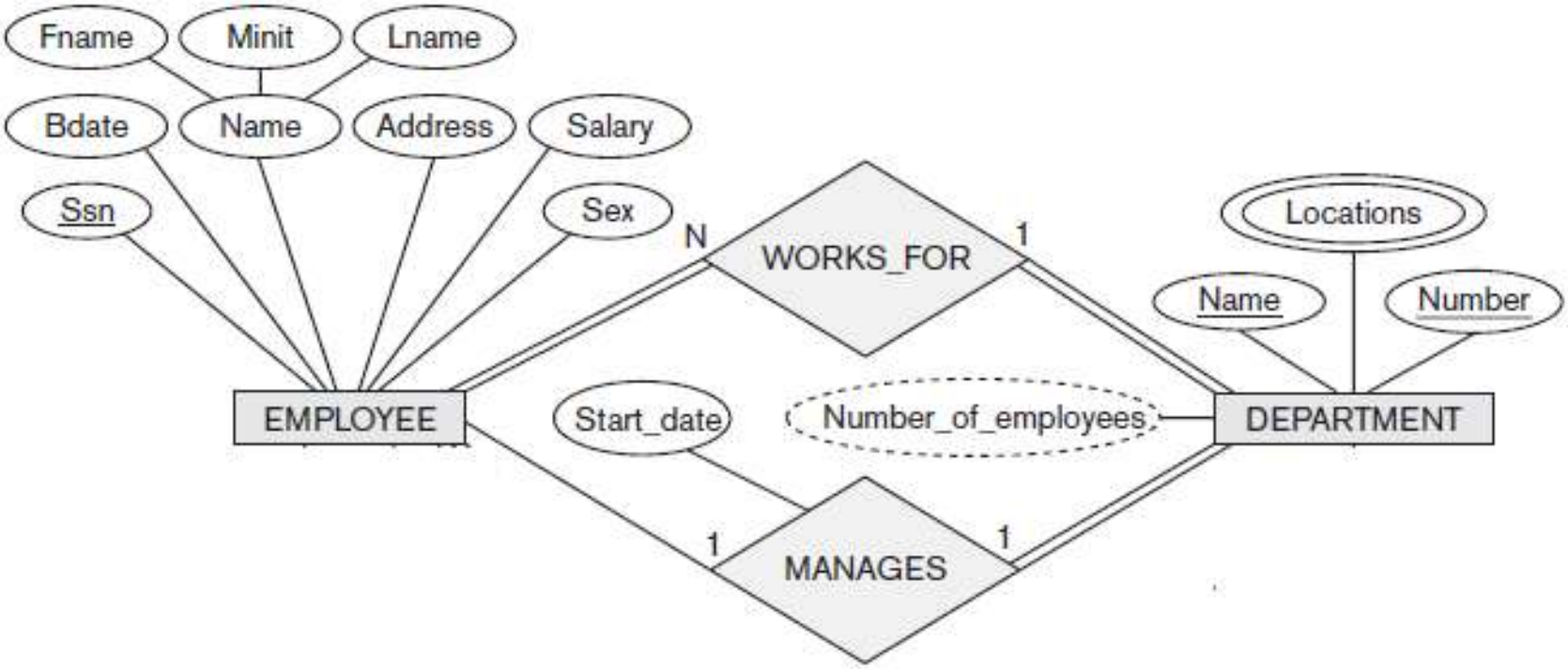
- A relationship type R among n entity types E1, E2,.. En defines a set of associations among entities from these entity types.
- Relationship instance: Each relationship instance ri in R is an association of entities, where the association includes exactly one entity from each participating entity type.
- ***For example, consider a relationship type WORKS_FOR between the two entity types EMPLOYEE and DEPARTMENT. Each relationship instance in the relationship set WORKS_FOR associates one EMPLOYEE entity and one DEPARTMENT entity.***



Relationship Types, Relationship Sets, Roles, & Structural Constraints



Relationship Types, Relationship Sets, Roles, & Structural Constraints

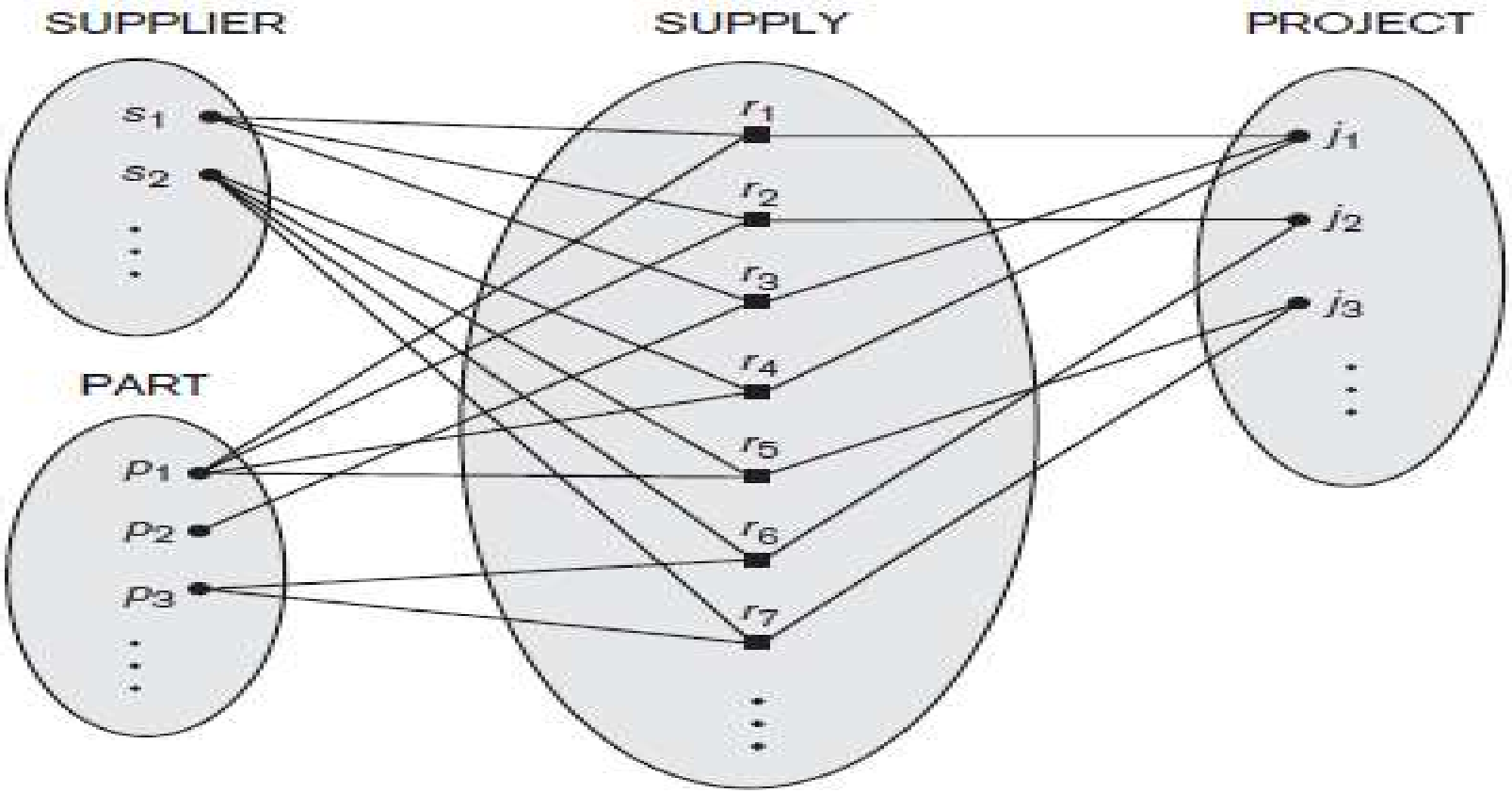


In ER diagrams, relationship types are displayed as **diamond-shaped boxes, which are connected by straight lines to the rectangular boxes representing the participating entity types. The relationship name is displayed in the diamond-shaped box.**

Relationship Degree, Role Names, and Recursive Relationships

Degree of a Relationship Type

- The degree of a relationship type is the number of participating entity types. Example the WORKS_FOR relationship is of degree two.



Relationship Degree, Role Names, and Recursive Relationships

- **Types of degree**

based on the number of linked entity types, we have 4 types of degrees of relationships.

1.Unary

2.Binary

3.Ternary

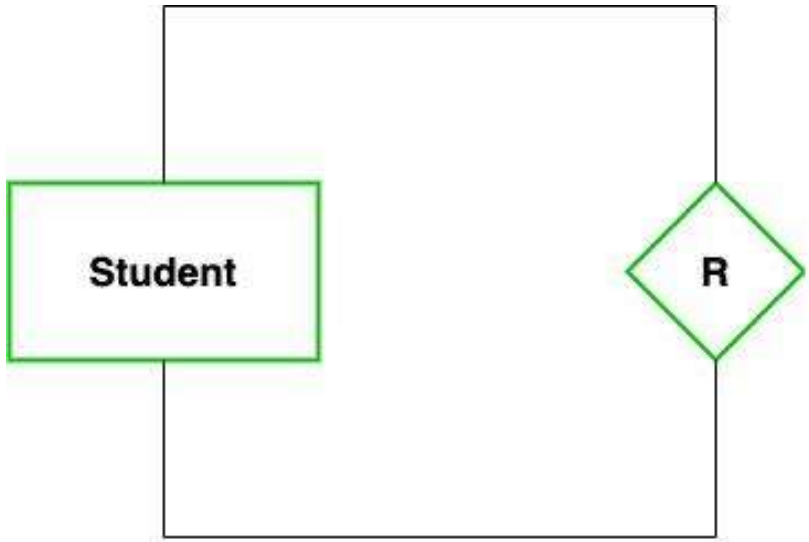
4.N-ary



Relationship Degree, Role Names, and Recursive Relationships

1. **Unary (Degree 1)**

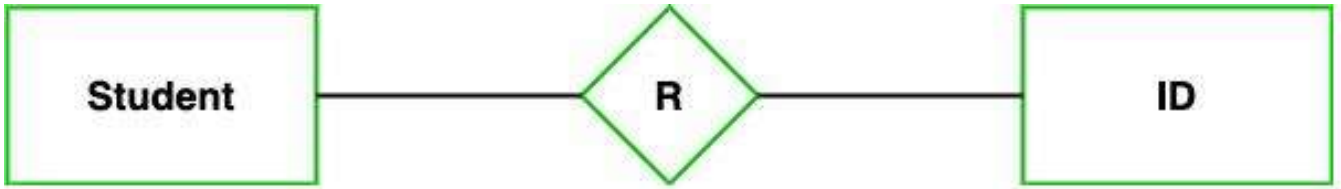
- In this type of relationship, both the associating entity types are the same. So, we can say that unary relationships exist when both entity types are the same and we call them the degree of relationship is 1. In other words, In a relation only one entity set is participating then such type of relationship is known as a unary relationship.



Relationship Degree, Role Names, and Recursive Relationships

2. Binary (Degree 2)

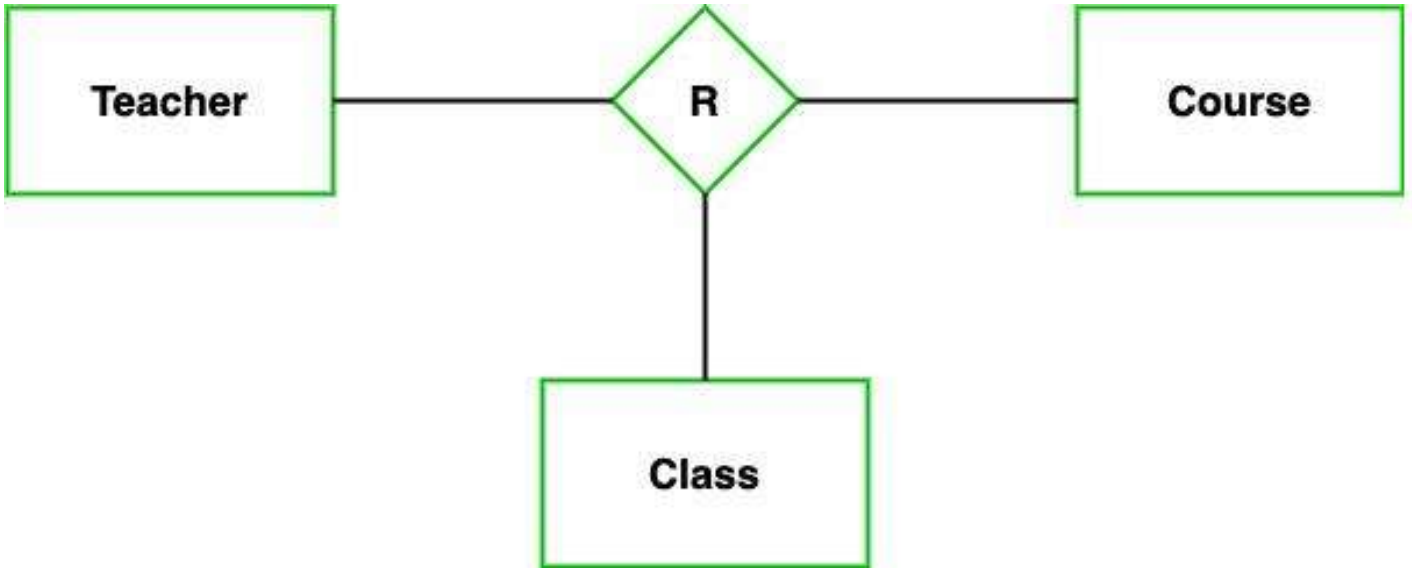
- In a relation when two entity sets are participating then such type of relationship is known as a binary relationship. This is the most used relationship and one can easily be converted into a relational table.



Relationship Degree, Role Names, and Recursive Relationships

3.Ternary

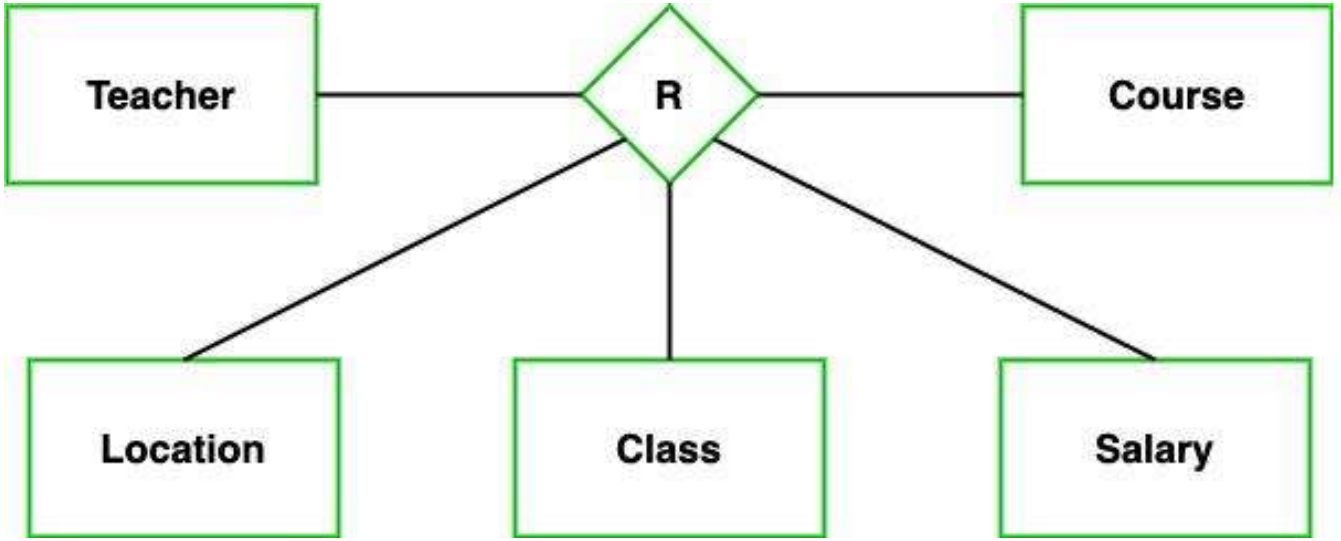
- In the Ternary relationship, there are three types of entity associates. So, we can say that a Ternary relationship exists when there are three types of entity and we call them a degree of relationship is 3



Relationship Degree, Role Names, and Recursive Relationships

4.N-ary(n-degree)

- In the N-ary relationship, there are n types of entity that associates. So, we can say that an N-ary relationship exists when there are n types of entities. There is one limitation of the N-ary relationship, as there are many entities so it is very hard to convert into an entity, rational table



Role Names and Recursive Relationships.

- Each entity type in a relationship plays a particular role. The **role name specifies the role that a participating entity type plays in the relationship and explains what the relationship means.**
- *For example, in the WORKS_FOR relationship type, EMPLOYEE plays the role of employee or worker and DEPARTMENT plays the role of department or employer.*
- ▪ However, in some cases the same entity type participates more than once in a relationship type in different roles. In such cases the role name becomes essential for distinguishing the meaning of the role that each participating entity plays. Such relationship types are called **recursive relationships.**



Constraints on Binary Relationship Types

- Relationship types have certain constraints that limit the possible combination of entities that may participate in relationship.
- There are two main types of relationship constraints
- **1.cardinality ratio**
- **2.participation.**



Constraints on Binary Relationship Types

1.cardinality ratio

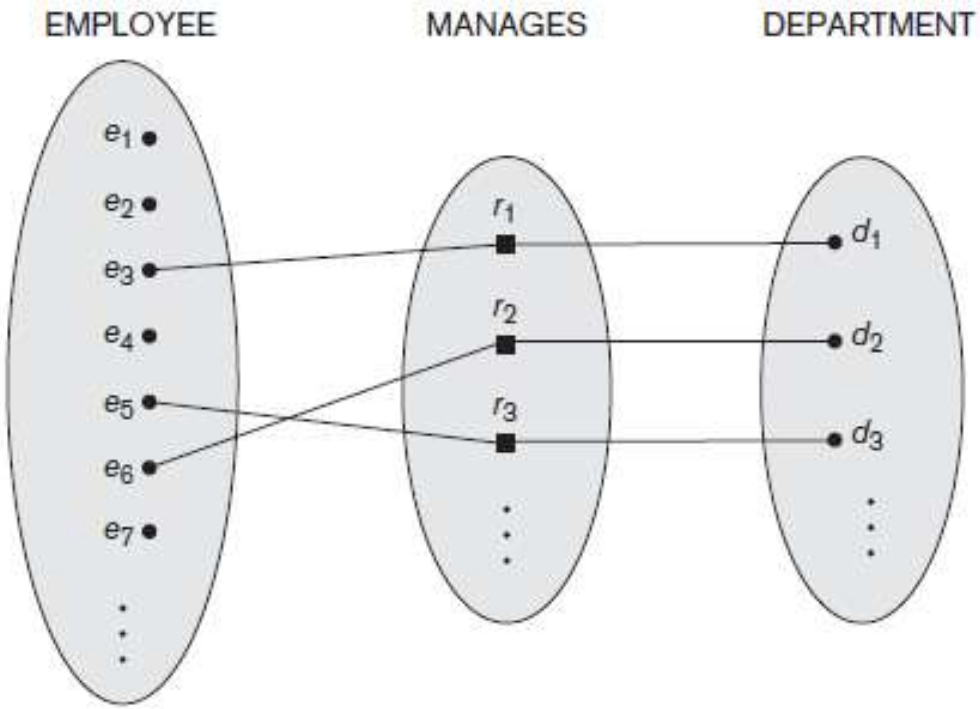
- The cardinality ratio specifies the maximum number of relationship instances that an entity can participate in.
- The possible cardinality ratios for binary relationship types are: **1:1, 1:N, N:1, M:N.**



Constraints on Binary Relationship Types

One-to-one – when only one instance of an entity is associated with the relationship, it is marked as '1:1'.

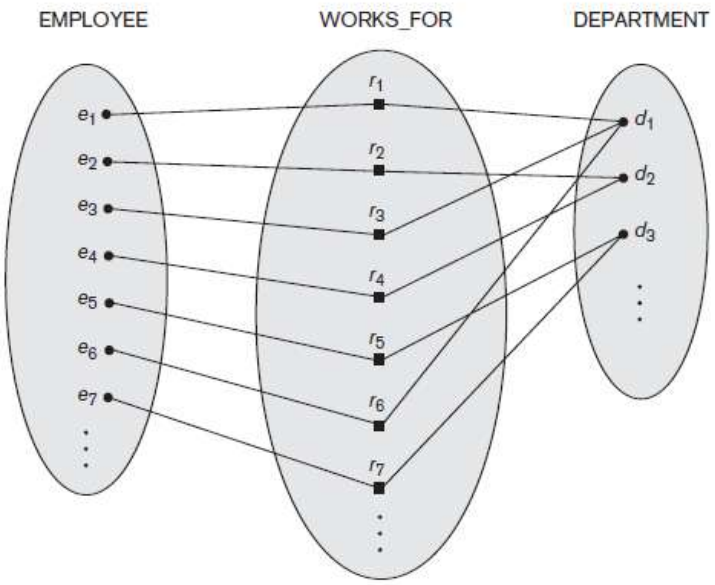
- *An example of a 1:1 binary relationship is MANAGES which relates a department entity to the employee who manages that department.*



Constraints on Binary Relationship Types

One-to-many − When more than one instance of an entity is associated with a relationship, it is marked as '1:N'.

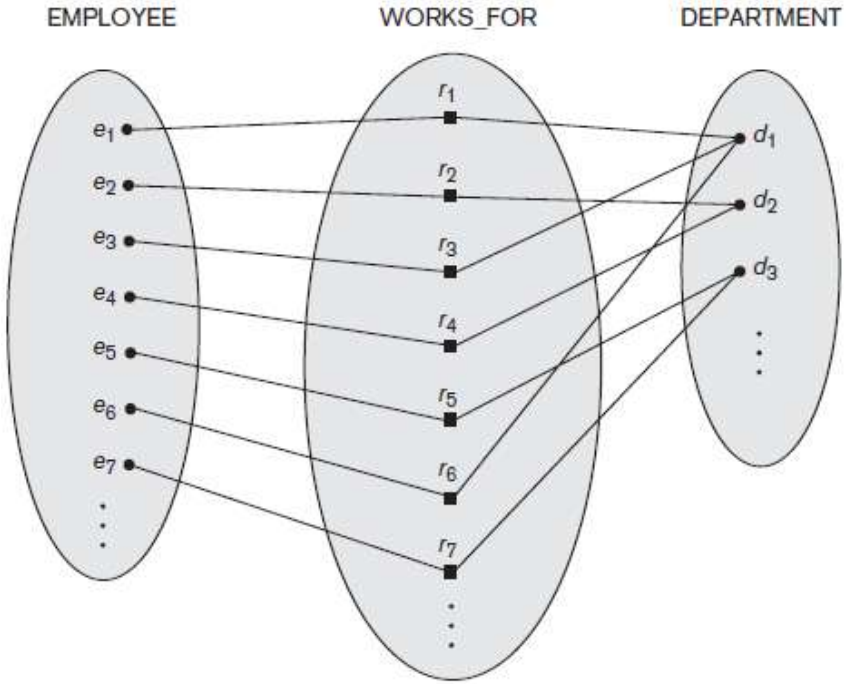
- *For example, in the WORKS_FOR binary relationship type, DEPARTMENT: EMPLOYEE is of cardinality ratio 1:N, meaning that each department can have any number of employees, but an employee work for only one department.*



Constraints on Binary Relationship Types

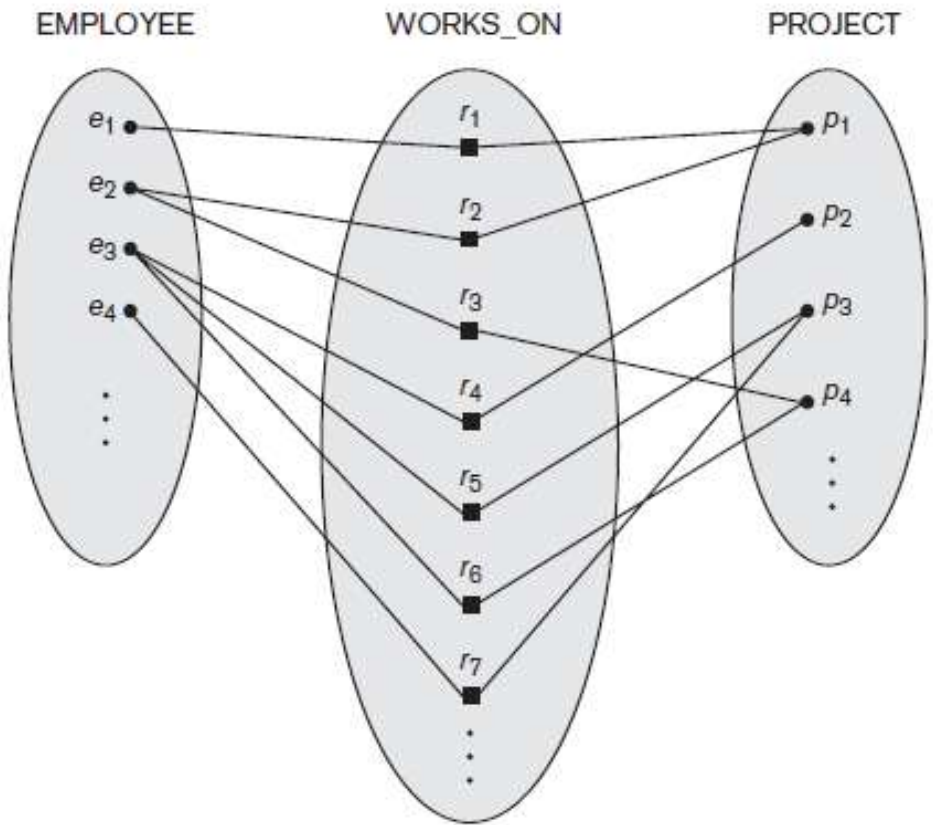
Many-to-one − When more than one instance of entity is associated with the relationship, it is marked as 'N:1'.

- *For example, in the WORKS_FOR binary relationship type, EMPLOYEE: DEPARTMENT is of cardinality ratio N:1, meaning that many number of employees work for one department.*



Constraints on Binary Relationship Types

Many-to- Many The relationship type WORKS_ON is of cardinality ratio M:N, an employee can work on several projects and in a project there might be several employees.



Participation Constraints and Existence Dependencies.

- The participation constraint specifies whether the existence of an entity depends on its being related to another entity via the relationship type.
- The constraint specifies the minimum number of relationship instances that each entity can participate in.
- There are two types of participation constraints:

1. Total Participation.

2. Partial Participation.

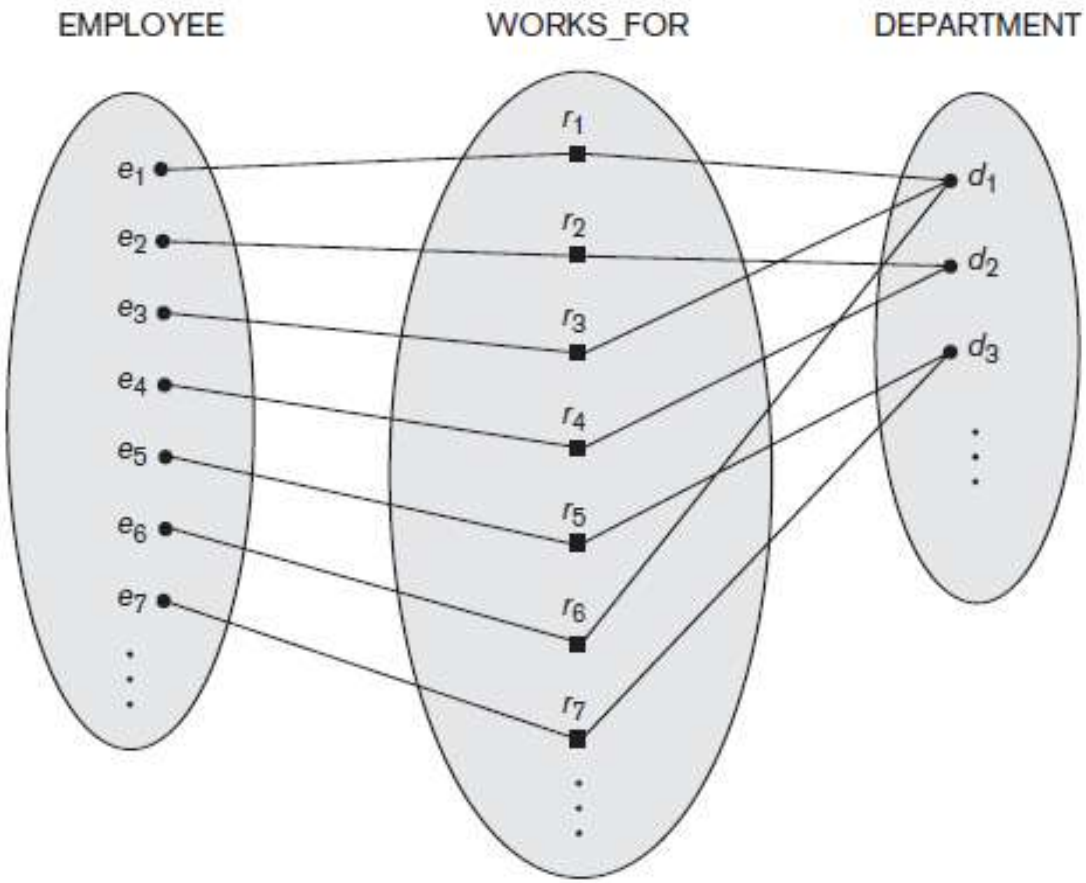


Participation Constraints and Existence Dependencies.

Total Participation is when each entity in the entity set occurs in at least one relationship in that relationship set.

- If a company policy states that *every employee must work for a department, then an employee entity can exist only if it participates in at least one WORKS_FOR relationship instance.*
- ***Thus, the participation of EMPLOYEE in WORKS_FOR is called total participation, meaning that every entity in the total set of employee entities must be related to a department entity via WORKS_FOR.***
- It is also sometimes called an **existence dependency.**
- Total participation is represented by a double line, going from the relationship to the dependent entity.

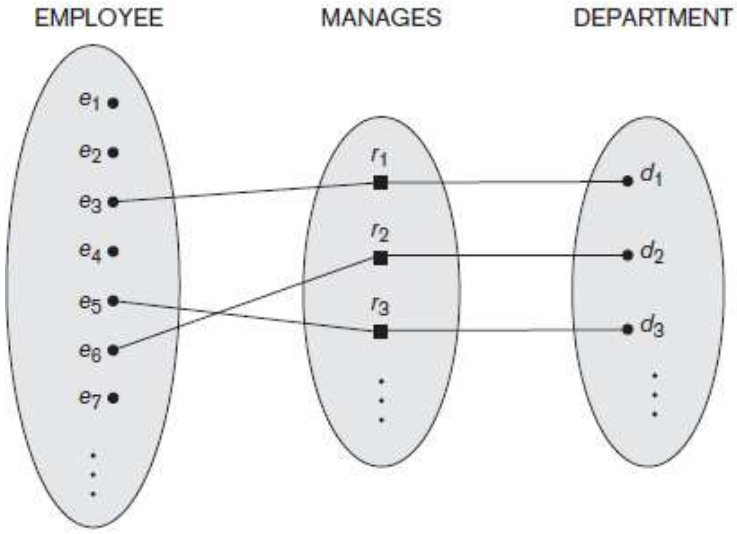
Participation Constraints and Existence Dependencies.



Participation Constraints and Existence Dependencies.

Partial Participation.

- If only a part of the set of entities participate in a relationship, then it is called **partial participation.**
- Using the Company example, every employee will not be a manager of a department, so the participation of an employee in the "Manages" relationship is partial.
- Partial participation is represented by a **single line.**

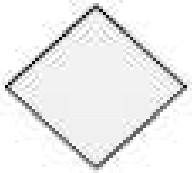
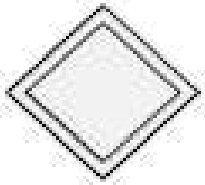


Weak Entity Types

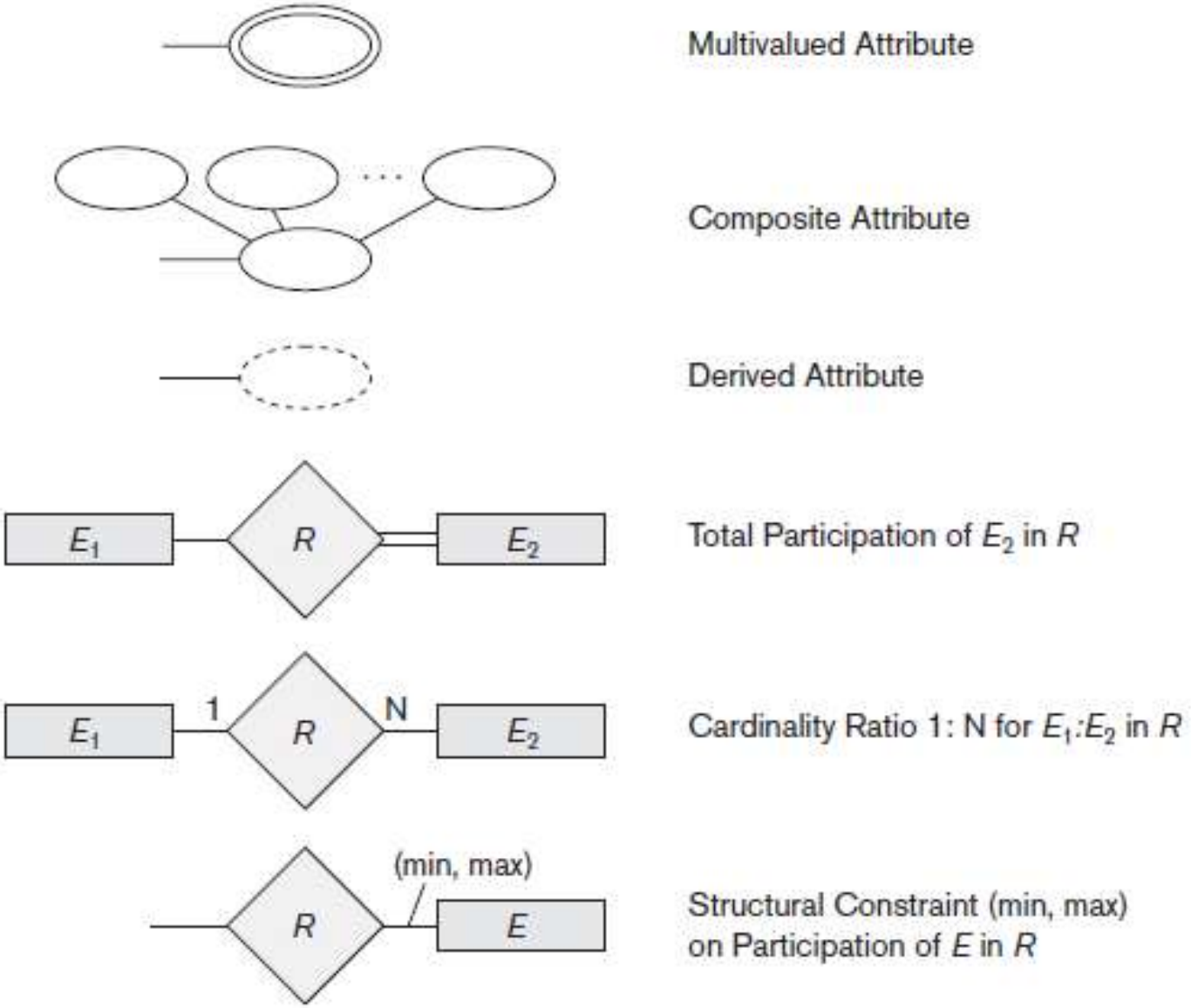
- Entity types that do not have key attributes are called **weak entity types.**
- Entities that belong to a weak entity type are identified by being related to specific entities from another entity type in combination with one of their attribute values. This entity type is called an **identifying or owner entity type.**



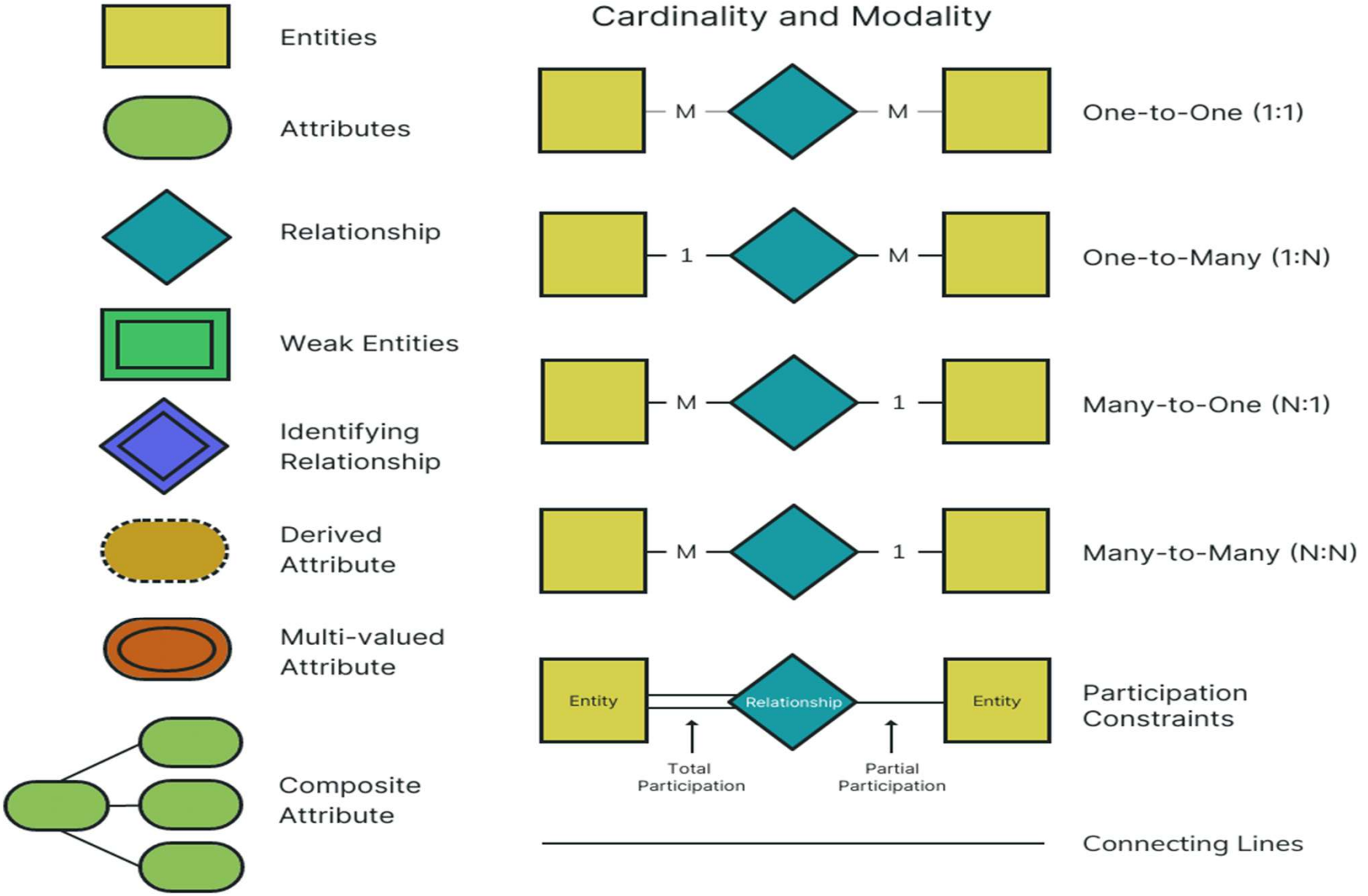
ER Diagrams, Naming Conventions, and Design Issues.

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Indentifying Relationship
	Attribute
	Key Attribute

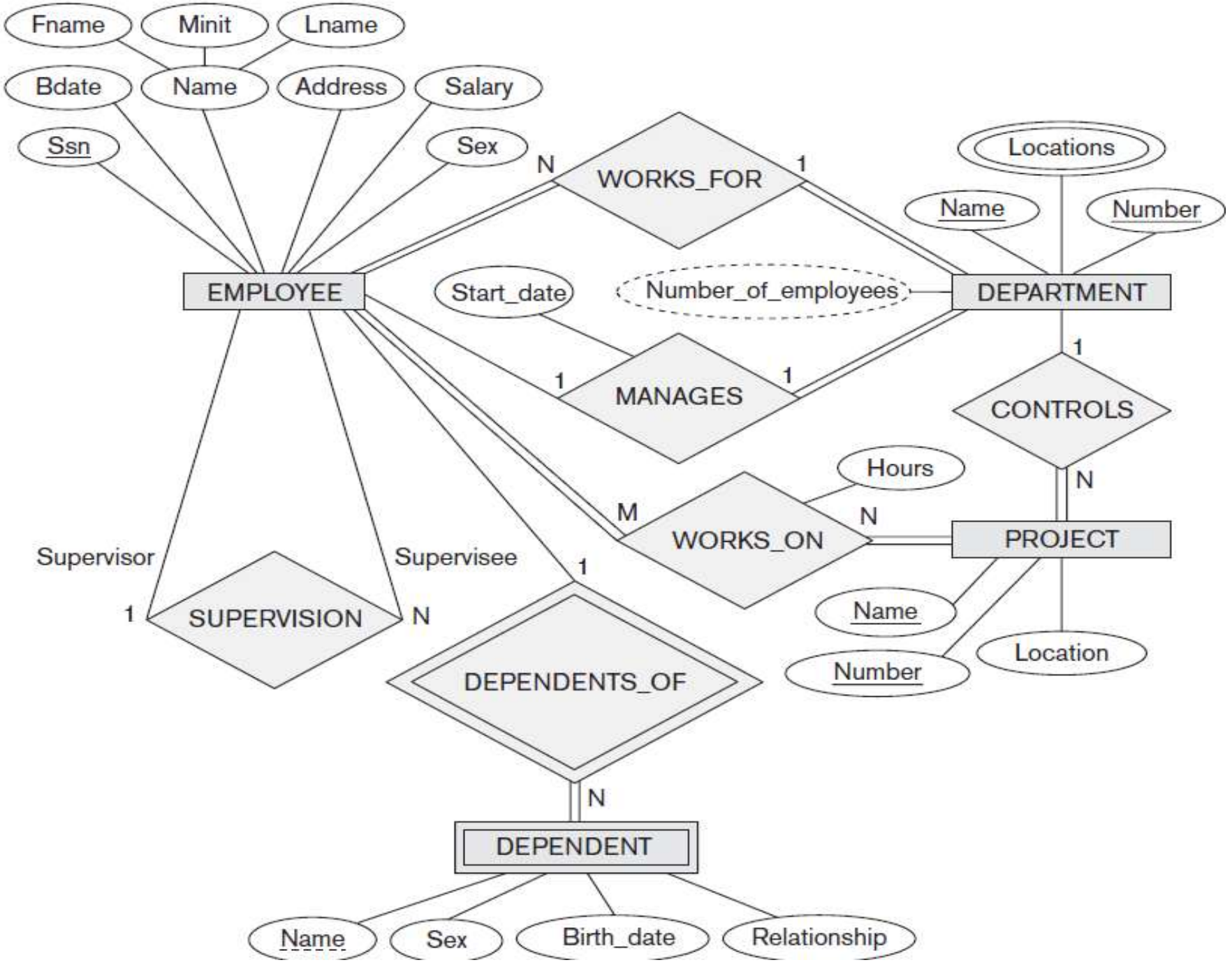
ER Diagrams, Naming Conventions, and Design Issues.



ER Diagrams, Naming Conventions, and Design Issues.



THE COMPANY DATABASE KEEPS TRACK OF A COMPANY'S EMPLOYEES, DEPARTMENTS, AND PROJECTS.



ER-Diagram Examples

Design an **ER Diagram for a Student Database System** used in a college.

Entities and Attributes

1. STUDENT: Student_ID (Primary Key), Name *(First_Name, Last_Name)*, Date_of_Birth Age *(Derived attribute)* Phone_Number *(Multivalued attribute)* Address *(Street, City, State, Zip)*

2. COURSE: Course_ID (Primary Key), Course_Name, Credits, Duration.

3. DEPARTMENT: Dept_ID (Primary Key). Dept_Name, Location. Phone_No

4. INSTRUCTOR: Instructor_ID (Primary Key), Name, Email, Qualification. Salary

5. ENROLLMENT: Enrollment_ID (Primary Key, Enrollment_Date, Grade

Relationships : Enrolls, Teaches, Offers, Belongs_To.

Student — Course

Instructor — Course

Department — Course

Student — Department



Entities and Attributes for Booking and Reservation Systems

1. **Users**: UserID(Primary Key), Name, Email, Phone.
2. **Services**: ServiceID(Primary Key),Name, Description, Price.
3. **Bookings**: BookingID(Primary Key),UserID, ServiceID, Date, Status.
4. **Locations**: LocationID(Primary Key), Name, Address, City, Country.
5. **Reviews**: ReviewID (Primary Key),UserID, ServiceID, Rating, Comment
6. **Payment**: PaymentID(Primary Key), BookingID, Amount, PaymentDate, PaymentMethod
7. **Cancellation/Refund**: CancellationRefundID(Primary Key), BookingID, CancellationDate, RefundAmount:
8. **Invoices**: InvoiceID(Primary Key), UserID, BookingID, Amount,Date.
9. **Notifications**: NotificationID(Primary Key),UserID, Message

KEYWORDS

- Makes
- Has
- Gives
- Have
- Receives



Relationships between Entities

1. User – Bookings Relationship
2. Services – Bookings Relationship
3. Users – Reviews Relationship
4. Services – Reviews Relationship
5. Services – Location Relationship
6. Bookings -Payment Relationship
7. Bookings – Cancellation/Refund Relationship
8. User – Payment Relationship
9. Users – Invoices Relationship
10. Users – Notifications Relationship



ER-Diagram Examples

Design an **ER Diagram for a Student Database System** used in a college.

Entities and Attributes

1. **STUDENT: Student_ID** (Primary Key) Name *(Composite → First_Name, Last_Name)*, Date_of_Birth Age *(Derived attribute)* Phone_Number *(Multivalued attribute)* Address *(Composite → Street, City, State, Zip)*
2. **COURSE: Course_ID** (Primary Key) Course_Name Credits Duration.
3. **DEPARTMENT: Dept_ID** (Primary Key) Dept_Name Location Phone_No.
4. **INSTRUCTOR: Instructor_ID** (Primary Key) Name Email Qualification Salary.
5. **ENROLLMENT: Enrollment_ID** (Primary Key) Enrollment_Date Grade

Relationships : Enrolls, Teaches Offers Belongs_To

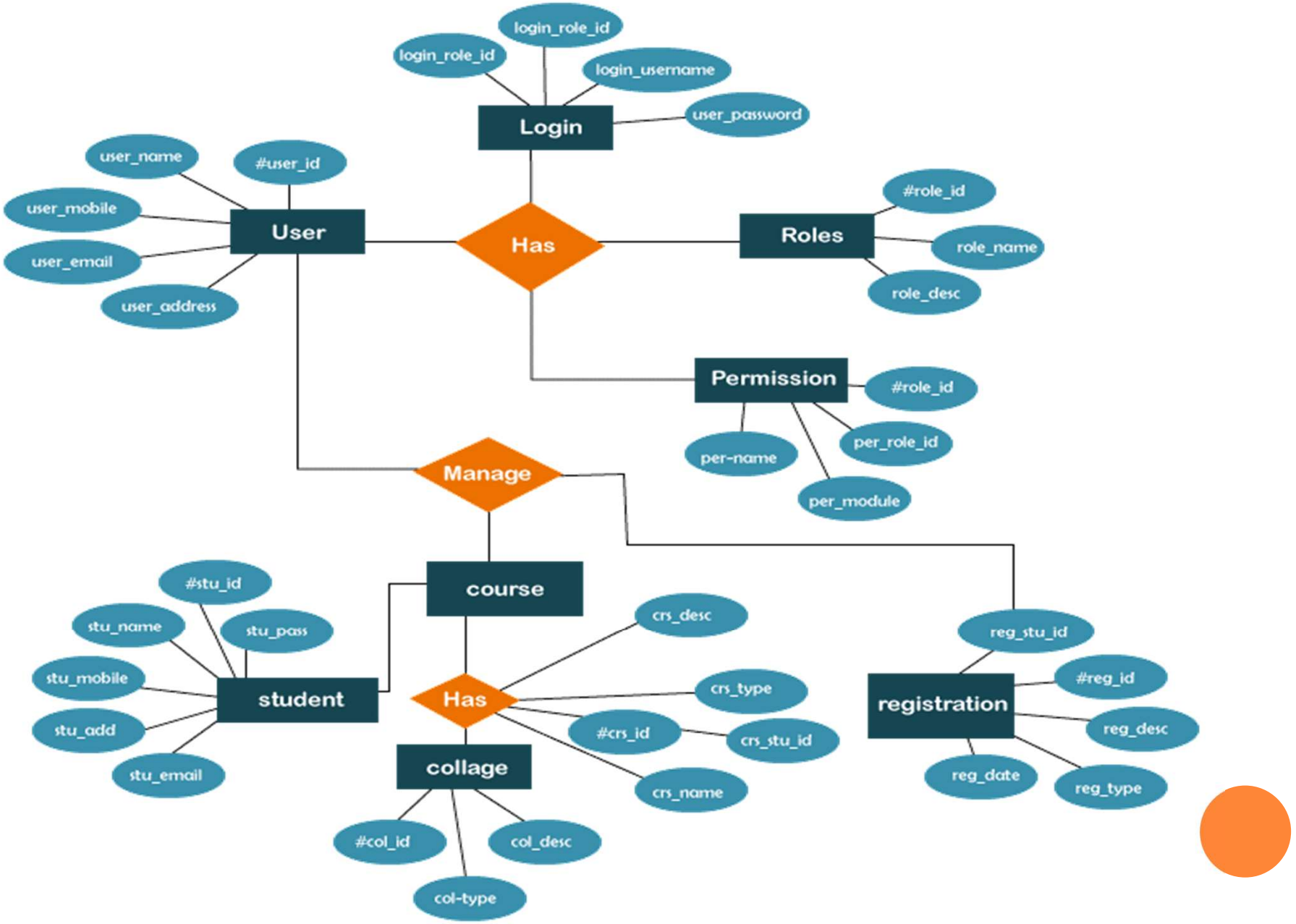
Student — Course

Instructor — Course

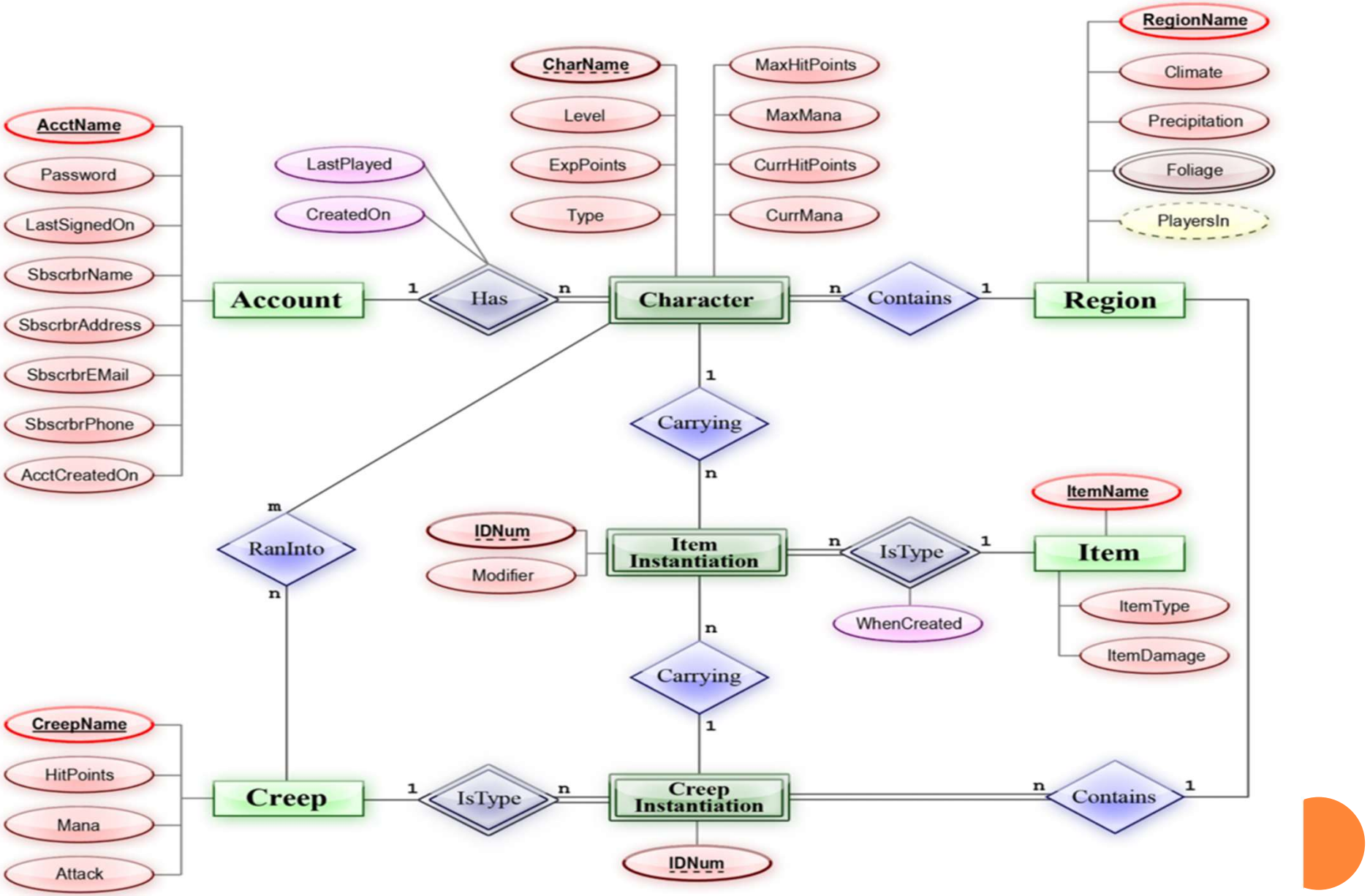
Department — Course

Student — Department

ER-Diagram


login_role_id
login_role_id
login_username
user_password
Login
user_name
#user_id
user_mobile
user_email
user_address
User
Has
Roles
#role_id
role_name
role_desc
Permission
#role_id
per-name
per_role_id
per_module
Manage
course
#stu_id
stu_name
stu_pass
stu_mobile
stu_add
stu_email
student
crs_desc
Has
crs_type
#crs_id
crs_stu_id
crs_name
collage
#col_id
col_desc
col-type
reg_stu_id
#reg_id
registration
reg_desc
reg_date
reg_type

ER-Diagram

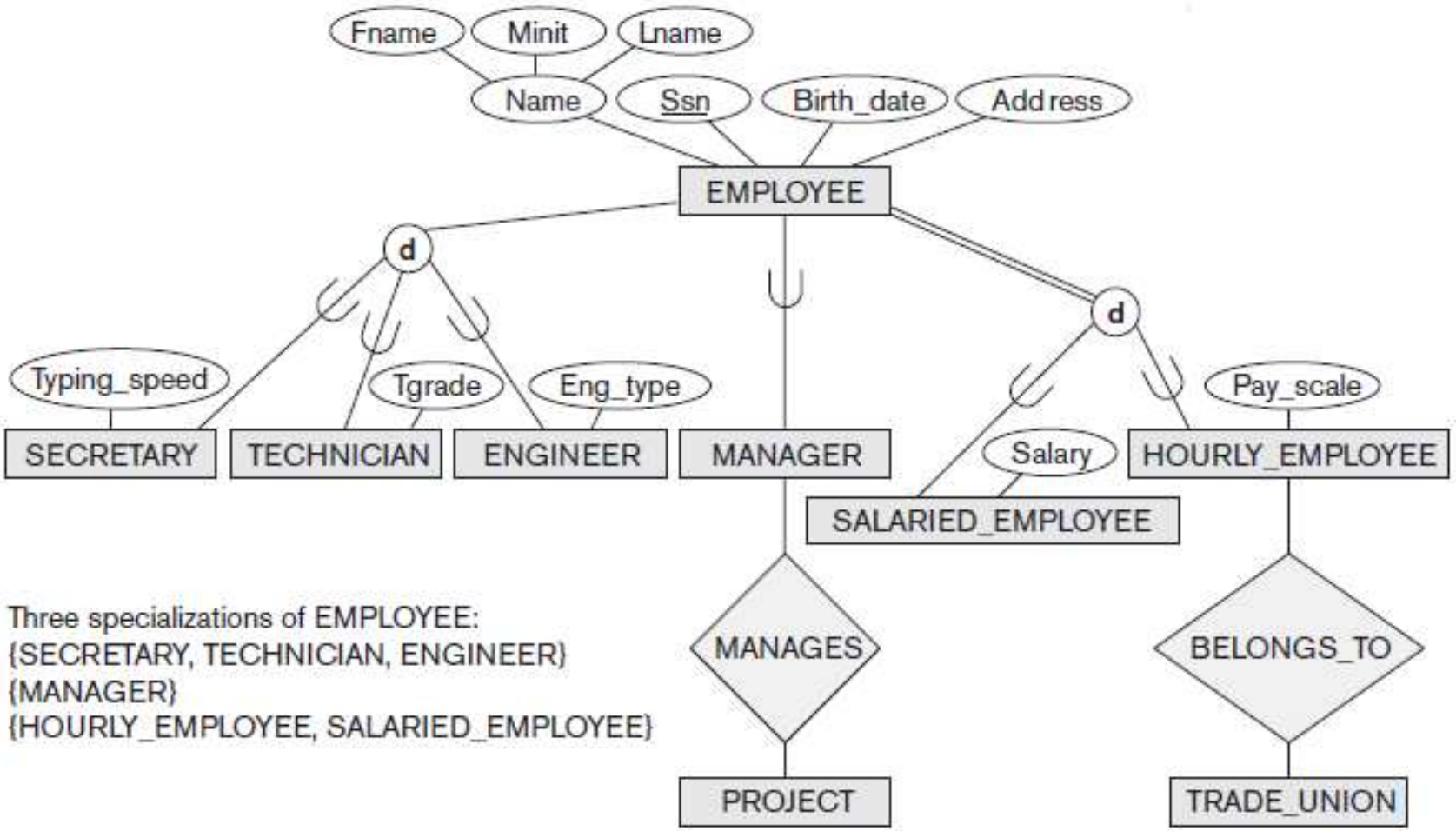

RegionName
Climate
Precipitation
Foliage
PlayersIn
CharName
Level
ExpPoints
Type
MaxHitPoints
MaxMana
CurrHitPoints
CurrMana
AcctName
Password
LastSignedOn
SbscrbrName
SbscrbrAddress
SbscrbrEMail
SbscrbrPhone
AcctCreatedOn
LastPlayed
CreatedOn
Account
1
Has
n
Character
n
Contains
1
Region
1
Carrying
n
m
RanInto
n
IDNum
Modifier
Item Instantiation
n
IsType
1
Item
WhenCreated
ItemName
ItemType
ItemDamage
n
Carrying
1
CreepName
HitPoints
Mana
Attack
Creep
1
IsType
n
Creep Instantiation
n
Contains
1
IDNum

Specialization and Generalization

- Specialization is the process of defining a set of subclasses of an entity type; this entity type is called the superclass of the specialization.
- The subclasses that define a specialization are attached by lines to a **circle that represents the specialization**

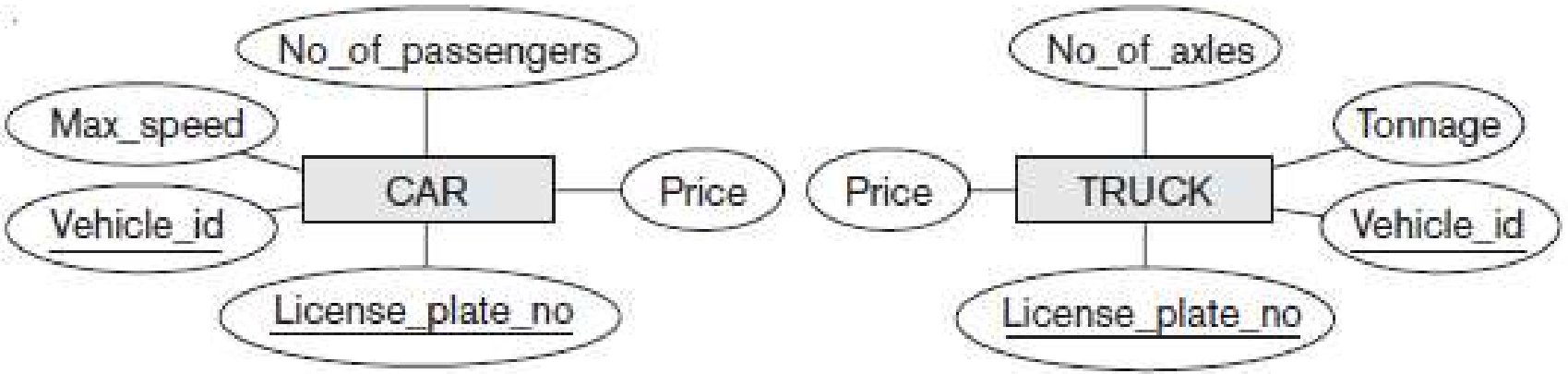


SPECIALIZATION AND GENERALIZATION



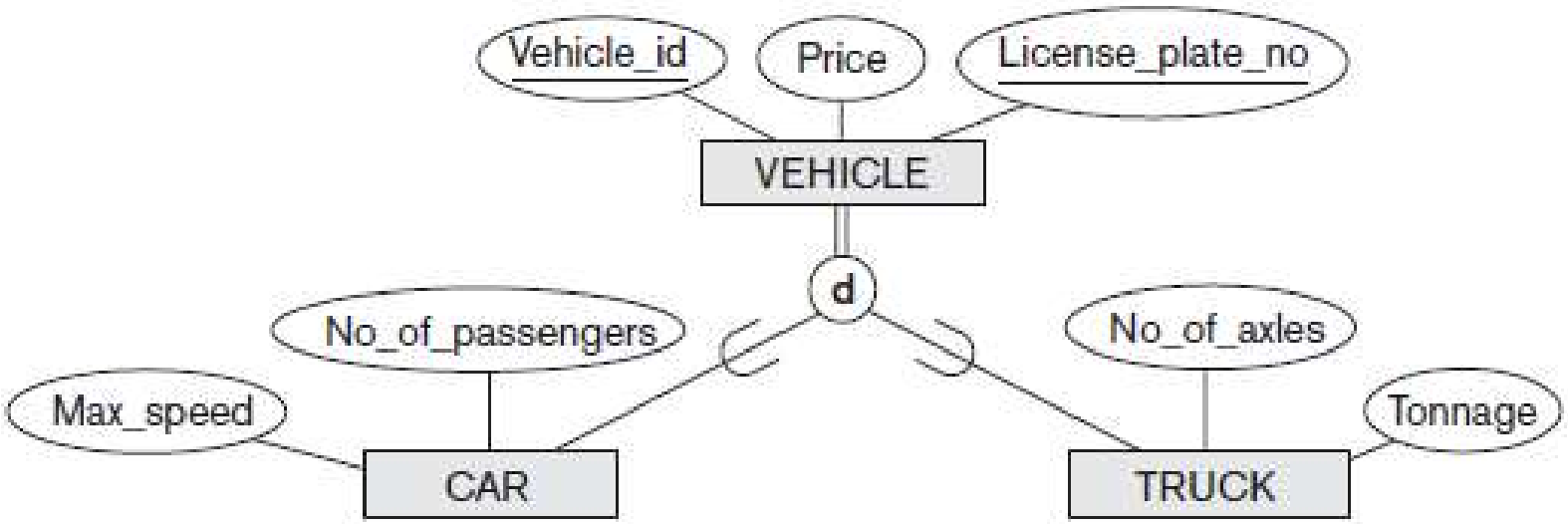
SPECIALIZATION AND GENERALIZATION

- The reverse process of abstraction suppresses the differences among entity types, identify their common features, and **generalize them into a single superclass of which the original entity types are special subclasses.**



Specialization and Generalization

- An arrow pointing to the generalized superclass represents a generalization, whereas arrows pointing to the specialized subclasses represent a specialization.



Relational database design using ER-to-Relational Mapping

Step 1: Mapping of Regular Entity Types

Step 2: Mapping of Weak Entity Types

Step 3: Mapping of Binary 1:1 Relationship

Step 4: Mapping of Binary 1:N Relationship

Step 5: Mapping of Binary M:N Relationship

Step 6: Mapping of Multivalued Attributes

Step 7: Mapping of N-ary Relationship Types

Step 8: Mapping Specialization / Generalization

Step 9: Mapping of Union Types (Category)



Relational database design using ER-to-Relational Mapping

Step 1: Mapping of Regular Entity Types

A **strong (regular) entity** becomes a **table**.

Example

Entity: **STUDENT** (Student_ID (Primary Key),Name, Age, Department)

Relational Schema

STUDENT(Student_ID, Name, Age, Department)

Primary Key: Student_ID

Sample Table

Student_ID	Name	Age	Department
S101	Rahul	20	CSE
S102	Ananya	21	ECE



Relational database design using ER-to-Relational Mapping

Step 2: Mapping of Weak Entity Types

A weak entity depends on a strong entity.

Example

Entity: EMPLOYEE (Strong)

Weak Entity: DEPENDENT

Attributes:

EMPLOYEE(Emp_ID (PK),Name)

DEPENDENT(Dep_Name, Age)

Relational Schema

EMPLOYEE(Emp_ID, Name)

Primary Key: Emp_ID

DEPENDENT(Emp_ID, Dep_Name, Age)

Primary Key: (Emp_ID, Dep_Name)

Foreign Key: Emp_ID → EMPLOYEE

Example Table

Emp_ID	Dep_Name	Age
E1	Riya	10
E1	Aryan	7



Relational database design using ER-to-Relational Mapping

Step 3: Mapping of Binary 1:1 Relationship

One entity is related to **only one entity**.

Example

Entities:

- PERSON
- PASSPORT

Relationship: **HAS**

Solution

Add **foreign key** to one table.

Relational Schema

PERSON(Person_ID, Name)

PASSPORT(Passport_No, Person_ID)

Foreign Key: Person_ID → PERSON

Example

Passport_No	Person_ID
P1001	101



Relational database design using ER-to-Relational Mapping

Step 4: Mapping of Binary 1:N Relationship

One entity relates to **many entities**.

Example

Entities:

- DEPARTMENT
- EMPLOYEE

Relationship: **WORKS_IN**

Relational Schema

Add **foreign key on N side**

DEPARTMENT(Dept_ID, Dept_Name)

EMPLOYEE(Emp_ID, Name, Dept_ID)

Foreign Key: Dept_ID → DEPARTMENT

Example

Emp_ID	Name	Dept_ID
E1	Ravi	D1
E2	Meena	D1

Relational database design using ER-to-Relational Mapping

Step 5: Mapping of Binary M:N Relationship

Many-to-many relationships require a **new relation**.

Example

Entities: STUDENT **ENROLLS** COURSE

Relational Schema

STUDENT(Student_ID, Name)

COURSE(Course_ID, Course_Name)

ENROLL(Student_ID, Course_ID)

Primary Key: (Student_ID, Course_ID)

Foreign Key: Student_ID → STUDENT

Foreign Key: Course_ID → COURSE

Example

Student_ID	Course_ID
S101	C201
S102	C201



Relational database design using ER-to-Relational Mapping

Step 6: Mapping of Multivalued Attributes

A **multivalued attribute** becomes a **separate table**.

Example

Entity: **STUDENT**

Multivalued attribute: **Phone_No**

Relational Schema

STUDENT(Student_ID, Name)

STUDENT_PHONE(Student_ID, Phone_No)

Primary Key: (Student_ID, Phone_No)

Foreign Key: Student_ID → STUDENT

Example

Student_ID	Phone_No
S101	9876543210
S101	9123456780



Relational database design using ER-to-Relational Mapping

Step 7: Mapping of N-ary Relationship Types

Relationship involving **more than two entities**.

Example

Entities:

- SUPPLIER
- PART
- PROJECT

Relationship: **SUPPLIES**

Relational Schema

SUPPLIER(Supplier_ID, Name)

PART(Part_ID, Part_Name)

PROJECT(Project_ID, Project_Name)

SUPPLIES(Supplier_ID, Part_ID, Project_ID, Quantity)

Primary Key: (Supplier_ID, Part_ID, Project_ID)



Relational database design using ER-to-Relational Mapping

Step 8: Mapping Specialization / Generalization

Inheritance between entities.

Example

Superclass: **EMPLOYEE**

Subclasses:

- MANAGER
- ENGINEER

Relational Schema

EMPLOYEE(Emp_ID, Name, Salary)

MANAGER(Emp_ID, Bonus)

Foreign Key: Emp_ID → EMPLOYEE

ENGINEER(Emp_ID, Skill)

Foreign Key: Emp_ID → EMPLOYEE

Example

EMPLOYEE

Emp_ID	Name	Salary
E1	Ravi	50000

MANAGER

Emp_ID	Bonus
E1	10000

Relational database design using ER-to-Relational Mapping

Step 9: Mapping of Union Types (Category)

A **category** represents a subset of multiple entity sets.

Example

Entities:

- PERSON
- COMPANY

Category:

- OWNER (can be person or company)

Relational Schema

PERSON(Person_ID, Name)

COMPANY(Company_ID, Name)

OWNER(Owner_ID)

Owner may reference either PERSON or COMPANY.



Relational database design using ER-to-Relational Mapping

Step	ER Concept	Mapping
1	Regular Entity	Table
2	Weak Entity	Table with owner key
3	1:1 Relationship	FK in one table
4	1:N Relationship	FK in N side
5	M:N Relationship	New table
6	Multivalued Attribute	Separate table
7	N-ary Relationship	New table
8	Specialization	Superclass + subclass tables
9	Category	Separate relation

